

TRANSLATION_PROVIDER

HelixLLM CPU-Capable Translation (NMT) Provider — Design Spike (P3-T5 / CPU tier)

Status	DESIGN (spike before implementation, §11.4.6 — do not code the provider until this is agreed)
Scope	The CPU-only professional-translation capability — NO GPU, NO VRAM-broker dependency, so it ships before the P0/P1 GPU chain
Owns	The CPU slice of implementation-plan item P3-T5 (04_implementation_plan.md line 83)
Created	2026-07-06 · Revision 1 · Track (T1/main) · Branch feature/helixllm-full-extension
Grounding	submodules/helix_llm/docs/API_CONTRACT.md §2/§4 · submodules/helix_llm/docs/VRAM_BROKER.md §2 (CPU-only tier) · docs/research/07.2026/00_master/ 04_implementation_plan.md P3-T5 · sibling EMBEDDINGS_PROVIDER.md (structure mirror)

Anti-bluff (§11.4.6): every RAM/latency/quality figure in this document is an **estimate to be measured** ((EST — measure)). None is a captured benchmark; each **MUST** be replaced by on-host RSS + p50/p95/p99 wall-clock + measured chrF/COMET on the project’s own fixtures under docs/qa/<run-id>/translation/ before any PASS. No fabricated benchmark appears below.

0. Why this can ship before the GPU tiers

The programme’s critical path (04_implementation_plan.md §“Critical-path sequencing”) hard-serialises **P0 host GPU foundation → P1 serving core + VRAM broker → P2 gateway** before the P3 extended capabilities, because the coder fleet, VLM, image/video generators, and the GPU translation-quality tier all contend for the single **RTX 5090 · 32 GB** (VRAM_BROKER.md §1).

The **CPU translation provider designed here takes no GPU reservation**. VRAM_BROKER.md §2 explicitly lists the CPU-only tier as “*Qdrant, HelixMemory, NLLB (CTranslate2 int8 ~3.5 GB — can also be CPU), Tesseract*” with “*no GPU reservation ... 0 GB GPU*”. The broker’s admission Class “translate” (VRAM_BROKER.md §4, §47) resolves to a **zero-byte VRAM lease** for this CPU variant. It therefore does **not** depend on P0-T1 GPU passthrough, the P0-T3 sm_120 builds, or the P1-T4 residency broker being implemented — the CPU tier is admissible even when the whole card is committed to the coder fleet.

Honest boundary vs the plan (§11.4.6).
04_implementation_plan.md:83 specifies P3-T5 as “**NLLB-200-3.3B** (CTranslate2 int8, warm) + **TOWER+ 9B** (vLLM, on-demand); LibreTranslate-compatible /translate + doc/glossary. Signature: COMET + back-translation metamorphic (guard COMET-gaming CX-05).”
The 3.3B NLLB + 9B TOWER+ are the GPU/warm-tier targets. This document honours the task directive to treat translation as **CPU-capable** by defining the CPU tier explicitly: the **ship-first CPU model is NLLB-200-distilled-600M** (with distilled-1.3B as the higher-quality CPU lane), behind the **same LibreTranslate-compatible /translate route** and the **same CTranslate2 engine** the GPU tier will use. The CPU → GPU migration is then a model-size + device change (600M CPU → 1.3B/3.3B GPU), not an engine or API change. This is a documentation reconciliation, flagged so a reviewer can confirm — not a contradiction.

1. Engine choice — evidence-based comparison

1.1 The candidates

#	Engine	Serves / translate (LibreTranslate-shape) natively?	CPU-first?	Auto-detect built in?	New runtime to the tree?
A	NLLB-200 via CTranslate2	No — CT2 is a library; needs a		No — pair with a detector	

#	Engine	Serves / translate (LibreTranslate- shape) natively?	CPU- first?	Auto-detect built in?	New runtime to the tree?
	int8 (distilled-600M / 1.3B)	thin LibreTranslate- shaped HTTP shim	Yes (int8 C++ CPU kernels)	(fastText lid.176 / stanza)	CT2 runtime + small shim container
B	Marian / OPUS- MT via CTranslate2 (Helsinki-NLP, per-pair)	Via stock LibreTranslate (Argos Translate → OpenNMT- CTranslate2 backend)	Yes (fastest of the four)	Yes (LibreTranslate source: "auto")	Stock LibreTranslate container (no new engine — same CT2)
C	TOWER-class LLM via llama.cpp (Tower+ 2B GGUF CPU / 9B GPU)	No — chat- framed; needs a / translate → chat adapter	2B: marginal on CPU; 9B: GPU- preferred	No — prompt- driven	llama.cpp already in- tree; GGUF weights new
D	Hosted-API fallback (DeepL / Google / OpenAI)	No — bespoke adapter	N/A (remote)	Provider-side	New remote dep + secret + egress

1.2 Decision

Primary engine: NLLB-200 via CTranslate2 int8 — ship-first model facebook/nllb-200-distilled-600M converted to CT2 int8 (upgrade lane: nllb-200-distilled-1.3B int8), fronted by a **thin LibreTranslate-compatible / translate HTTP shim** wrapping ctranslate2.Translator + SentencePiece + a language detector.

Documented fallback lane: stock LibreTranslate (option B — Argos Translate’s OpenNMT-CTranslate2 + OPUS/Argos per-pair models), because it serves the **identical / translate route**, runs on the **same CTranslate2 engine** (no new runtime), and ships auto-detect + /languages + /detect **out of the box** — a genuine zero-new-API degraded path when a language pair or the NLLB weights are unavailable.

1.3 Justification (cited, LATEST upstream docs — §11.4.99 / §11.4.150)

1. **NLLB-200 CT2 int8 is exactly what the plan names, and it is genuinely CPU-capable.** The pre-converted OpenNMT checkpoint documents `compute_type=int8` for CPU inference (`int8_float16` is the GPU variant), “*Speedup inference while reducing memory by 2x–4x using int8 inference in C++ on CPU or GPU*”, `ctranslate2>=3.22.0`, FLORES-200 language codes (`eng_Latn`, `spa_Latn`), SentencePiece tokenisation, converted with `quantization="int8"`. NLLB covers **200 languages in one model** — a single resident CPU process serves every pair, unlike OPUS’s one-model-per-pair. Sources: [OpenNMT/nllb-200-distilled-1.3B-ct2-int8 \(HF model card\)](#) (accessed 2026-07-06); [OpenNMT/nllb-200-distilled-1.3B-ct2-int8 upload commit 70f572a](#) (accessed 2026-07-06); [CTranslate2 \(OpenNMT\) — supports Transformer/M2M-100/NLLB/BART/mBART on CPU+GPU with int8 quantisation](#) (accessed 2026-07-06).
2. **CTranslate2 is the fastest, lowest-memory CPU inference engine of the set, and covers BOTH NLLB and OPUS-MT.** Independent benchmarking reports CT2 “*outperformed other libraries like Marian and Transformers, generating more tokens per second while using less memory*” and that int8 quantisation makes models “*4× smaller and 4× faster without significantly sacrificing quality*” — the exact CPU-throughput lever the CPU tier needs. One engine serves the primary (NLLB) and the fallback (OPUS-MT) lane. Source: [CTranslate2: Anti-Internet Translation — Naufal Pratama, May 2026 \(benchmark\)](#) (accessed 2026-07-06).
3. **Stock LibreTranslate is a real, no-new-API fallback on the same engine.** LibreTranslate is built on **Argos Translate**, whose `.argosmodel` packages contain “*an OpenNMT CTranslate2 model, a SentencePiece tokenisation model, a Stanza tokenizer ... and metadata*” — i.e. the fallback lane is the identical CT2 runtime with OPUS/Argos per-pair weights, and it already exposes the exact `/translate` contract the plan names (see §2), including `source:"auto"` auto-detect, `/detect`, `/languages`, and multi-language pivoting. Sources: [LibreTranslate — Translate Text API operation](#) (accessed 2026-07-06); [argos-translate README \(CTranslate2 + SentencePiece + Stanza backend\)](#) (accessed 2026-07-06).
4. **Why OPUS-MT is the fallback, not the primary.** OPUS-MT (Marian, Helsinki-NLP) is the **fastest** option but the **lowest quality** — “*the Opus-MT translation result is not as good as NLLB and M2M ... a trade-off ... speed at the expense of quality*” — and needs a **separate model per language pair** (a packaging + coverage burden vs NLLB’s single 200-language model). It is the ideal degraded lane (light, drop-in, same route) but not the professional-quality primary. Source: [CTranslate2 benchmark \(Naufal Pratama, May 2026\) — OPUS-MT fastest but lower BLEU than NLLB/M2M](#) (accessed 2026-07-06).

5. **Why a TOWER-class LLM is a later upgrade, not the CPU primary.**

Unbabel’s **Tower+** open-weight family (2B/9B/72B) is genuinely SOTA for translation + instruction-following (glossary/formality obedience) and 2B **GGUF exists for llama.cpp** — but the 9B is the plan’s **GPU on-demand** tier, the 2B is only marginal on CPU for bulk work, and llama.cpp translation needs a /translate → chat adapter. Tower+ 2B GGUF (llama.cpp) is documented here as the **CPU LLM-quality alternative** that bridges to the plan’s existing Tower+ GPU tier (no new ecosystem), NOT the ship-first CPU default. Sources: [Unbabel/Tower-Plus-9B \(open-weight 2B/9B/72B\)](#) (accessed 2026-07-06); [Tower+ paper arXiv:2506.17080](#) (accessed 2026-07-06); [tensorblock/Unbabel Tower-Plus-9B-GGUF \(llama.cpp-compatible\)](#) (accessed 2026-07-06).

6. **Why the hosted-API lane (D) is explicitly rejected for the default path.** A hosted translator (DeepL/Google/OpenAI) requires a committed API key (§CONST-042 / §11.4.10 secret-leak surface) + third-party data egress of user-supplied text, and defeats the offline/self-host posture that makes this the ship-before-GPU capability. It MAY be added later as an **opt-in, config-injected, key-from-.env** provider descriptor, never the default.

1.4 CPU model selection (ship-first) — measured, not asserted

Engine fixed; the **model is a config value** injected per §CONST-046 / §11.4.35, never hardcoded. Candidate CPU models, all CTranslate2-runnable:

Model	Params	Languages	Quality	License	Role
facebook/nllb-200-distilled-600M (CT2 int8)	600M	200 (single model)	good	CC-BY-NC-4.0	Default CPU model — best coverage/size trade
facebook/nllb-200-distilled-1.3B (CT2 int8)	1.3B	200 (single model)	better	CC-BY-NC-4.0	Higher-quality CPU lane (heavier)
Helsinki-NLP/opus-mt-<src>-<tgt> (CT2)	~75M/pair	1 pair each	lower, fastest	mixed (mostly CC-BY / Apache)	Fallback lane — light, per-pair, drop-in
	2B		high		CPU LLM lane +

Model	Params	Languages	Quality	License	Role
Unbabel/Tower-Plus-2B (GGUF, llama.cpp)		20+ pairs, instruction-following		see model card	bridge to GPU Tower+ tier

License note (§11.4.6, flag for review): NLLB weights are **CC-BY-NC-4.0 (non-commercial)**. If HelixLLM ships translation in a commercial context, the license **MUST** be reconciled (Open Question Q5) — the OPUS-MT fallback lane (mostly permissive) or a permissively-licensed model becomes load-bearing. This is a documentation flag, not a resolution.

Recommendation: default nllb-200-distilled-600M CT2 int8 — 200-language coverage in one resident CPU process, int8 CPU-comfortable, FLORES-200 codes; the 1.3B is the drop-in quality lane (same engine, same route); OPUS-MT is the “fits anywhere / a pair NLLB handles badly” fallback; Tower+ 2B GGUF is the LLM-quality-follows-glossary lane that pre-stages the GPU Tower+ tier.

1.5 CPU RAM / latency / quality budget — ESTIMATES to be measured (§11.4.6)

All figures (EST — measure); replace with on-host RSS + p50/p95/p99 + measured chrF/COMET under docs/qa/<run-id>/translation/ before any PASS:

Model	Weights (int8) (EST)	Resident RSS incl. runtime (EST)	Single short-sentence latency (EST)	Batched throughput (EST)	Quality vs GPU tier (EST)
OPUS-MT (per pair, ~75M)	~80 MB	~0.4–0.8 GB	~15–60 ms	high	lower (fallback)
NLLB-distilled-600M	~600 MB–1 GB	~1.5–2.5 GB	~60–250 ms	med	good
NLLB-distilled-1.3B	~1.3–1.7 GB	~2.5–3.8 GB (matches broker’s ~3.5 GB CPU note)	~150–500 ms	med-low	better
Tower+ 2B GGUF (llama.cpp Q4/Q5)	~1.3–1.9 GB	~2–3.5 GB	~0.5–3 s (autoregressive)	low	high (glossary/formality)

Host has **64 cores / 251 GiB** (RESUME.md live-state), so even the 1.3B CPU model is comfortably resident with room for CT2 batch buffers; the CPU tier never competes with the GPU coder fleet for the 32 GB card. Threads / inter_threads / intra_threads are the tuning + observer-effect bound (§11.4.128-adjacent hygiene). **These numbers gate nothing until measured.**

2. API contract — LibreTranslate-compatible / translate, HelixLLM-consistent placement

The plan names a **LibreTranslate-compatible / translate** contract (04_implementation_plan.md:83). HelixLLM today registers no such route (API_CONTRACT.md §2 route inventory) — this provider **adds** it. To stay consistent with the shipped gateway (API_CONTRACT.md §2/§3: /v1 group with API-key middleware router.go:63-64, mandatory TLS 1.3 on :8443, OpenAI-error envelope), the route is registered **under the gateway /v1 group** as POST /v1/translate with the **LibreTranslate request/response body shape** — the plan's LibreTranslate payload, placed where every other authenticated HelixLLM capability lives, so it inherits API-key auth + rate-limit + security-headers uniformly.

Consistency note (§11.4.6): bare /translate (like /ws, /metrics) would be **unauthenticated** under the present wiring (API_CONTRACT.md §9 finding). Registering /v1/translate inside the gateway group is the reviewed choice so the route is API-key-authed. Whether to ALSO expose a bare /translate alias for drop-in LibreTranslate-client compatibility is Open Question Q1.

2.1 Request — LibreTranslate /translate body

```
POST /v1/translate
Authorization: Bearer <key>           # gateway /v1 API-key middleware (r
Content-Type: application/json
{
  "q": ["The cat sat on the mat.", "Quarterly revenue rose four percent."],
  "source": "auto",                    # ISO code (e.g. "en") OR "auto" →
  "target": "es",
  "format": "text",                    # "text" (default) | "html" (protec
  "alternatives": 0,                   # OPTIONAL – N alternative translat
  "model": "helix-translate"           # OPTIONAL Helix alias → backing NL
}
```

- q — string or []string (LibreTranslate batch shape).

- **source** — an ISO-639 code, or **"auto"** to trigger language detection (§2.3). Internally mapped to FLORES-200 (en → eng_Latn, es → spa_Latn) for the NLLB lane; passed through directly for the OPUS/LibreTranslate lane.
- **target** — required ISO code → FLORES-200 for NLLB.
- **format** — "text" | "html"; "html" protects markup and is the mechanism for **do-not-translate spans** (§5).
- **model** — a **Helix alias**, not a raw HF id (CONST-036/037: models come from the provider layer / LLMsVerifier, never hardcoded).

Source for the request/response contract: [LibreTranslate — Translate Text API operation](#) (accessed 2026-07-06) and [LibreTranslate API usage guide](#) (accessed 2026-07-06).

2.2 Response — LibreTranslate /translate body

```
{
  "translatedText": ["El gato se sentó en la alfombra.", "..."],
  "detectedLanguage": [
    {"language": "en", "confidence": 98.4}
  ],
  "alternatives": []
}
```

- **translatedText** — string for a scalar q, [] string for an array q (mirrors the q shape).
- **detectedLanguage** — present when source: "auto"; {language, confidence} (confidence 0–100 float), matching LibreTranslate exactly.
- **alternatives** — present when alternatives>0.

2.3 Source/target language handling + auto-detect

- **Explicit source** (source: "en"): the shim maps en → eng_Latn, target → FLORES-200, and calls Translator.translate_batch with the target language token as the decoder prefix (NLLB's target-language mechanism).
- **Auto-detect** (source: "auto"): a CPU language-ID pass runs FIRST (fastText lid.176 or the LibreTranslate/stanza detector), the top language + confidence populate detectedLanguage, and its FLORES-200 code drives the NLLB call. On the fallback lane, LibreTranslate's own source: "auto" handles this natively (its detector + Argos pivoting).
- **GET /v1/languages** MUST advertise the supported language set (LibreTranslate /languages shape: [{code, name, targets[]}]) so clients and LLMsVerifier discover coverage (CONST-036).
- **POST /v1/detect** (LibreTranslate /detect shape) MAY be exposed for detect-only clients (Open Question Q1).

2.4 Model aliasing + provider registration

- A config-driven ProviderDescriptor (mirrors P2-T4, 04_implementation_plan.md:73) registers the CPU translation endpoint with LLMsVerifier so the helix-translate alias, its language coverage, and its translation capability flag are **verifier-sourced, not hardcoded** (CONST-036/037/040).
- Alias → backing-model map (helix-translate → NLLB-600M / 1.3B / OPUS-pair / Tower+2B) lives in HelixLLM config (env / YAML), never a source literal (§CONST-046). Adding a model or a language pair = a config edit, no code change.

2.5 Error shape

OpenAI-error envelope, consistent with the gateway (API_CONTRACT.md §3, auth.go:58-64): {"error": {"message": "...", "type": "invalid_request_error"}} for auth/validation (unsupported pair, empty q); a **503** with the same envelope when the backing container is not yet warm (honest “warming”, **never** an untranslated passthrough of q — that is the identity bluff §4 must kill).

3. Containerization — rootless podman via the containers submodule, NO GPU

Per §11.4.76 (containers-submodule mandate) + §11.4.161 (rootless runtime) the service boots **through** vasic-digital/containers (pkg/boot / pkg/compose / pkg/health), never a hand-run podman/docker command, and never rootful.

3.1 Image + run shape (illustrative — config-injected, no hardcoded host §CONST-045)

- **Primary (NLLB CT2) image:** a small image = python:3.x-slim + ctranslate2
 - sentencepiece + a FastAPI LibreTranslate-shaped shim + a CPU language detector. Selected linux/amd64 vs linux/arm64 by uname -m (§11.4.81 cross-platform parity), pinned by digest in production (§11.4.76 clause 2).
- **Fallback image:** the upstream libretranslate/libretranslate container (Argos/CTranslate2, CPU), booted via the same compose profile — zero-new-code degraded lane.
- **NO GPU:** the run spec contains **no** --device nvidia.com/gpu=all and no GPU security flag (contrast the P0 GPU proof

- 04_implementation_plan.md:44). This is the structural guarantee it needs no P0. compute_type=int8 (CPU), never int8_float16 (GPU).
- **Model source:** --model points at a CT2-converted NLLB dir (or the OPUS/Argos packages) on a persistent cache volume, OR a pre-fetched local model mounted read-only where \$MODELS_DIR is **injected** (env), never a literal. Weights are a §11.4.77 re-obtain artefact (gitignored; fetch_models.sh-class script downloads from HF + runs the CT2 conversion — matches the P7-T1 fetch_models.sh; the CT2 conversion command is the regeneration mechanism).
 - **Port:** a config-injected host port (distinct from the coder fleet's :18434 and the embeddings tier's :18435 in RESUME.md), reached by the HelixLLM gateway; --network per the containers-submodule compose spec.
 - **Boot is part of the test entry point** (§11.4.76 on-demand-infra invariant): the HelixQA translation bank boots the container via the submodule, waits on pkg/health (LibreTranslate /languages or a shim /health), then drives the gateway route — a short-circuit fake that skips the boot is a §11.4 violation.
 - **Broker interaction:** Acquire(ctx, "translate") (VRAM_BROKER.md §4) returns a **0-byte lease** for this CPU variant; the broker records the service as CPU-only tier and takes no GPU reservation, so it is admissible even with the whole card committed to the coder fleet. No broker code is required to ship the CPU provider (the broker is a P1 component; the 0-VRAM class is its CPU-tier default).
 - **Catalogue-Check (§11.4.74):** extend vasic-digital/containers@<sha> — add a translation compose profile (NLLB-shim + LibreTranslate) to the containers submodule; never an in-project ad-hoc compose file.

3.2 Cross-platform + resource hygiene

- ARM64 host → the linux/arm64 image variant chosen by runtime uname dispatch (§11.4.81); CT2 supports aarch64 CPU.
- Bounded CT2 threads (inter_threads/intra_threads) + a container memory limit (§12.3 container hygiene) so the translator never starves the developer host (§12.6); limits are config-injected and captured as evidence during acceptance.

4. Anti-bluff acceptance — the ONE machine-checkable runtime signature (§11.4.108)

Definition of done for this provider: on a **clean deploy** (§11.4.108/§11.4.139 — container freshly booted via the containers submodule, gateway pointed at it), the following single machine-checkable signature verifies and is captured to docs/qa/<run-id>/translation/:

RUNTIME SIGNATURE (translation forward-quality + back-translation metamorphic). For each pair in a tiny **golden reference set** (e.g. $en \rightarrow es$, $en \rightarrow fr$, $en \rightarrow de$, each a handful of source sentences with a human/golden reference translation), POST source S to POST `/v1/translate` and obtain forward translation F ; then POST F back (target $\rightarrow$ source) to obtain back-translation B . Assert **ALL** of: 1. **Not-identity / real-translation:** $F \neq S$ (normalised) AND the detected language of F matches the requested target — kills the copy/passthrough and the 1536-zero-vector-analogue “warming passthrough” bluff. 2. **Forward adequacy vs golden:** $chrF(F, golden_reference) \geq floor$ ($chrF$ is deterministic, reference-based, language-agnostic; floor calibrated on the project’s own fixtures per §11.4.107(13)). 3. **Back-translation metamorphic:** semantic similarity $sim(B, S) \geq margin$ ($chrF$ or embeddings-cosine via the CPU embeddings tier), the plan’s metamorphic relation. 4. **Determinism:** identical request $\rightarrow$ byte-identical translatedText across two calls (§11.4.50). The captured artefact is the raw `/v1/translate` JSON (forward + back) + the computed $chrF$ /similarity table + PASS/FAIL verdict with its evidence path.

Guard against COMET-gaming / metamorphic-gaming (plan guard CX-05).

Back-translation alone is gameable: an **identity copy** round-trips perfectly ($B==S$ because $F==S$), so criterion 3 in isolation would PASS a translator that does nothing. Criterion 1 ($F \neq S$ + target-language match) defeats that; criterion 2 (forward $chrF$ vs an independent golden reference) defeats a translator that emits *fluent-but-wrong* text that happens to round-trip. The **three criteria together** are the anti-gaming construction — no single one is trusted. Where the GPU tier later adds a COMET model (XLM-R, CPU-runnable but heavier), COMET is an **additional** forward-quality signal, never a replacement for the $chrF$ + not-identity + metamorphic triple.

4.1 Golden-good / golden-bad self-validation (§11.4.107(10))

The $chrF$ /metamorphic **analyzer itself is mutation-proofed** with a fixture set, wired into the meta-test:

- **golden-good fixture** — a captured real `/v1/translate` response where a genuine $en \rightarrow es \rightarrow en$ round-trip holds ($F \neq S$, target=es, $chrF \geq floor$, $sim \geq margin$) $\rightarrow$ the analyzer **MUST** return **PASS**.
- **golden-bad fixtures** (each **MUST** return **FAIL**, proving the analyzer cannot be fooled):
 1. **identity / passthrough** response (`translatedText == q`, the “warming passthrough” shape) $\rightarrow$ fails criterion 1 ($F==S$).
 2. **wrong-language** response (fluent text in the wrong target language) $\rightarrow$ fails the detected-target check + $chrF$ -vs-golden.

3. **garbage / shuffled** response (tokens shuffled, or a fixed canned string)
→ fails chrF-vs-golden AND the back-translation sim margin.
4. **empty** translatedText → fails all forward criteria.

Paired §1.1 mutation: strip the not-identity **OR** the chrF-vs-golden **OR** the metamorphic assertion from the analyzer → the identity/garbage golden-bad fixture PASSes → the gate FAILs. That mutation is the mechanical proof the acceptance test is not itself a bluff (and is the CX-05 anti-gaming guard made mechanical).

4.2 Higher-order + resilience proofs (compose, do not replace §4)

- **Auto-detect correctness:** POST source: "auto" for a known-language sentence → detectedLanguage.language matches truth with confidence above a fixture-calibrated floor.
- **Determinism / re-runability** (§11.4.50/§11.4.98): identical input → byte-identical output; the whole bank PASSes at -count=3.
- **Stress + chaos** (§11.4.85): batch of $N \geq 100$ sentences (throughput + p50/p95/p99 captured), $N \geq 10$ concurrent callers (no deadlock/leak), boundary inputs (empty string, max-length doc, unicode, an all-punctuation string, a string already in the target language) each categorised; chaos = container SIGKILL mid-request → gateway returns an honest 503 warning, **never** an untranslated passthrough.
- **Feature-class evidence (§11.4.69):** the closed sink-side taxonomy has no translation class today; this provider **adds one** (taxonomy is *open to additions, never contraction*) — evidence shape = the captured forward-chrF + not-identity + back-translation-metamorphic artefact above. Flagged for the §11.4.69 taxonomy owner.

4.3 Four-layer verification (§11.4.108)

1. **SOURCE** — /v1/translate route + shim + provider descriptor committed; pre-build grep gate.
 2. **ARTIFACT** — the NLLB CT2 int8 model dir present + the shim image pulled/pinned by digest; pkg/health green.
 3. **RUNTIME-ON-CLEAN-TARGET** — the §4 runtime signature verifies against a freshly-booted container (not a stale one) — the definition of done.
 4. **USER-VISIBLE** — a client (HelixCode / any CLI agent / a LibreTranslate client) POSTs /v1/translate, gets a real translation in the right language, and a downstream consumer (doc/glossary flow) uses it.
-

5. Professional-translation notes (design considerations)

Professional translation is more than raw NMT; these are design considerations, not v1 blockers (each maps to a lane above):

- **Do-not-translate (DNT) spans + terminology protection.** NLLB CT2 has no native glossary. Two mechanisms: (a) **format: "html"** — wrap DNT spans / product names / code in markup that LibreTranslate/the shim preserves untouched (LibreTranslate documents HTML-format tag preservation); (b) **placeholder masking** — replace terms with sentinel tokens (`[[T0]]`), translate, then restore. Both are pre/post-processing in the shim, deterministic, testable.
 - **Glossary / terminology enforcement.** For *enforced* target terminology (not just protection), the **Tower+ 2B (llama.cpp) lane** is load-bearing — Tower+ follows glossary + instruction constraints natively (its instruction- following is the paper's headline). The plan's "doc/glossary" P3-T5 scope maps to the Tower+ tier; the NLLB CPU default ships DNT-protection first, enforced glossary via the LLM lane.
 - **Formality control.** NLLB has no formality register control. Options: OPUS-MT formality-tuned variants where they exist, or the Tower+ instruction lane ("translate formally"). v1 CPU default: document the limitation honestly (§11.4.6) rather than fake a formality knob.
 - **Document / segment handling.** Sentence-boundary segmentation (stanza in the Argos lane; a splitter in the NLLB shim) so long documents translate segment-by- segment with context windows respected; re-assemble preserving whitespace/markup.
 - **Quality gating for professional output.** The §4 chrF + back-translation signature is the automated floor; a cometkiwi-style **reference-free QE** score (Unbabel) is the natural per-request confidence signal to surface to professional users when the GPU tier lands — flagged, not yet shipped. Source: [Unbabel COMET — reference-based + reference-free QE metrics](#) (accessed 2026-07-06).
-

6. Open questions (resolve before coding)

- **Q1** Register only POST /v1/translate (authed), or ALSO a bare / translate
 - /detect + /languages alias for drop-in LibreTranslate-client compatibility? (Bare routes are unauthenticated under current wiring — API_CONTRACT.md §9.)

- **Q2** Ship-first CPU model — nllb-200-distilled-600M (recommended) vs 1.3B vs the stock-LibreTranslate/OPUS lane as the *default* rather than the fallback. Decide on measured chrF/COMET + latency, not the \$1.5 estimates.
 - **Q3** Does the containers submodule already have a translation / LibreTranslate compose profile, or is this a §11.4.74 extend PR? (Investigate before scaffolding.)
 - **Q4** Language detector for source: "auto" — fastText lid.176 (fast, 176 langs) vs the LibreTranslate/stanza detector vs NLLB's own LID head. Consistency across the primary + fallback lanes matters.
 - **Q5 License** — NLLB is CC-BY-NC-4.0 (non-commercial). Reconcile the ship license against HelixLLM's distribution model, or make the permissively-licensed OPUS/Tower+ lane the commercial default. Blocking for any commercial ship.
 - **Q6** COMET model on CPU for the §4 signature — ship chrF-only first (light, deterministic) and add cometkiwi QE when the GPU tier lands, or run a small COMET on CPU now (heavier)? (Leaning: chrF + metamorphic now; COMET later.)
-

7. Composition footer — constitutional anchors touched

- **§11.4.6** (no-guessing) — every RAM/latency/quality figure flagged (EST – measure); the VRAM_BROKER CPU-tier reconciliation, the NLLB license, and the plan's 3.3B-vs-600M model choice all flagged, not asserted.
- **§11.4.74** (extend-don't-reimplement) — reuse CTranslate2 (the engine the plan already names) + LibreTranslate (the /translate contract the plan names) + extend the containers submodule; no bespoke NMT server or API.
- **§11.4.76 / §11.4.161** (containers submodule / rootless) — boot via pkg/boot+pkg/compose+pkg/health, rootless podman, no GPU device.
- **§11.4.77** (re-obtain mechanism) — model weights gitignored + fetch_models.sh (download + CT2-int8 conversion = the regeneration mechanism).
- **§11.4.81** (cross-platform parity) — linux/amd64 vs linux/arm64 chosen by runtime dispatch.
- **§11.4.99 / §11.4.150** (latest-source + deep multi-angle research) — engine decision cited to LATEST upstream docs across ≥4 distinct angles (NLLB/CT2, LibreTranslate/Argos, OPUS-MT benchmark, Tower+, COMET).
- **§11.4.107(10)/(13)** (self-validated analyzer + fixture-calibrated thresholds) — golden-good/golden-bad chrF+metamorphic analyzer, project-calibrated floor/margin; the CX-05 anti-gaming triple made mechanical.
- **§11.4.108 / §11.4.139** (four-layer runtime-signature on a clean target) — the §4 acceptance signature is the definition of done.

- **§11.4.69** (sink-side evidence taxonomy) — adds a translation feature class.
- **§11.4.85 / §11.4.98 / §11.4.50** (stress+chaos / full-automation / determinism).
- **§11.4.135** (standing regression guard) — the §4 signature registers as a permanent guard.
- **CONST-036/037/040** (LLMsVerifier single source of truth; capability flags verifier-sourced) — helix-translate alias + language coverage + translation capability from the verifier, never hardcoded.
- **CONST-042 / §11.4.10** (no-secret-leak) — the hosted-API lane rejected as default precisely because it requires a committed key + third-party egress.
- **CONST-046** (no hardcoded content) — model ids / host / port / pair map config-injected.

Sources verified

Deep-research 2026-07-06: - <https://huggingface.co/OpenNMT/nllb-200-distilled-1.3B-ct2-int8> - <https://huggingface.co/OpenNMT/nllb-200-distilled-1.3B-ct2-int8/commit/70f572adafa4794890ce7826156a4209717855af> - <https://github.com/OpenNMT/CTranslate2> - <https://docs.libretranslate.com/api/operations/translate/> - https://docs.libretranslate.com/guides/api_usage/ - <https://github.com/argosopentech/argos-translate> - <https://mprtmma.medium.com/ctranslate2-anti-internet-translation-69f0faea17d8> - <https://huggingface.co/Unbabel/Tower-Plus-9B> - <https://arxiv.org/abs/2506.17080> - https://huggingface.co/tensorblock/Unbabel_Tower-Plus-9B-GGUF - <https://github.com/Unbabel/COMET>

(Negative finding, §11.4.99(B): the canonical DeepL/Google/OpenAI translation-API reference pages were not fetched — the hosted-API lane is explicitly rejected as a default per CONST-042/§11.4.10, so its precise request shape is out of scope for this CPU-first design. The LibreTranslate / translate request/response shape in §2 is grounded in the LibreTranslate operations doc above, which is the authoritative contract THIS system targets per 04_implementation_plan.md:83.)